Class 20: Edge AI Systems

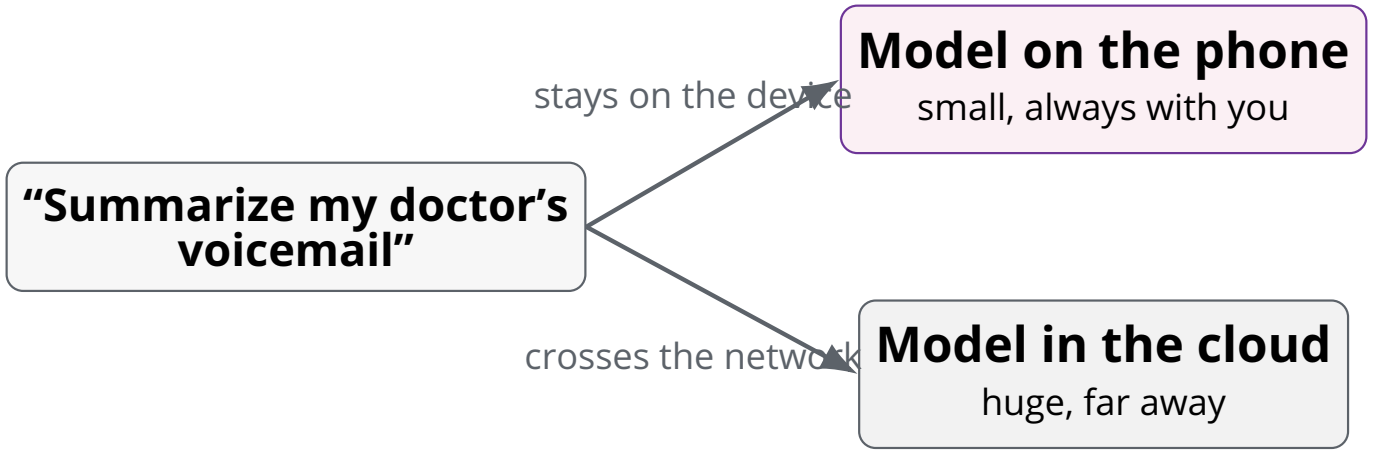
Where Should the Model Run?

Xin Liu

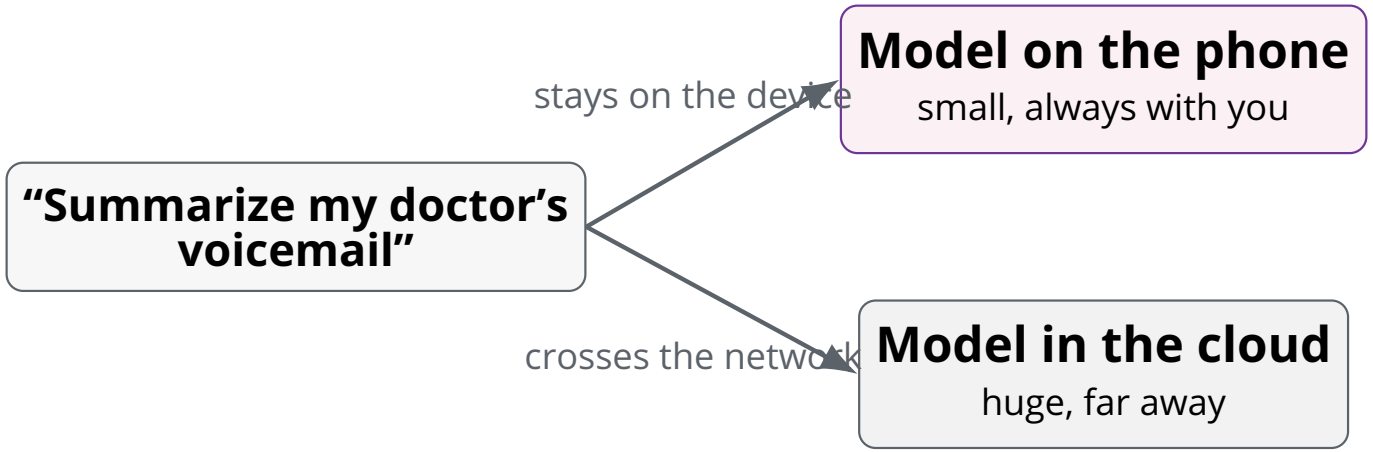
Florida State University
xliu15@fsu.edu

CIS 4930/5930 Future Edge Networks
xinliulab.github.io/FSU-CIS4930-CIS5930-Future-Edge-Networks

One question, two places to answer it

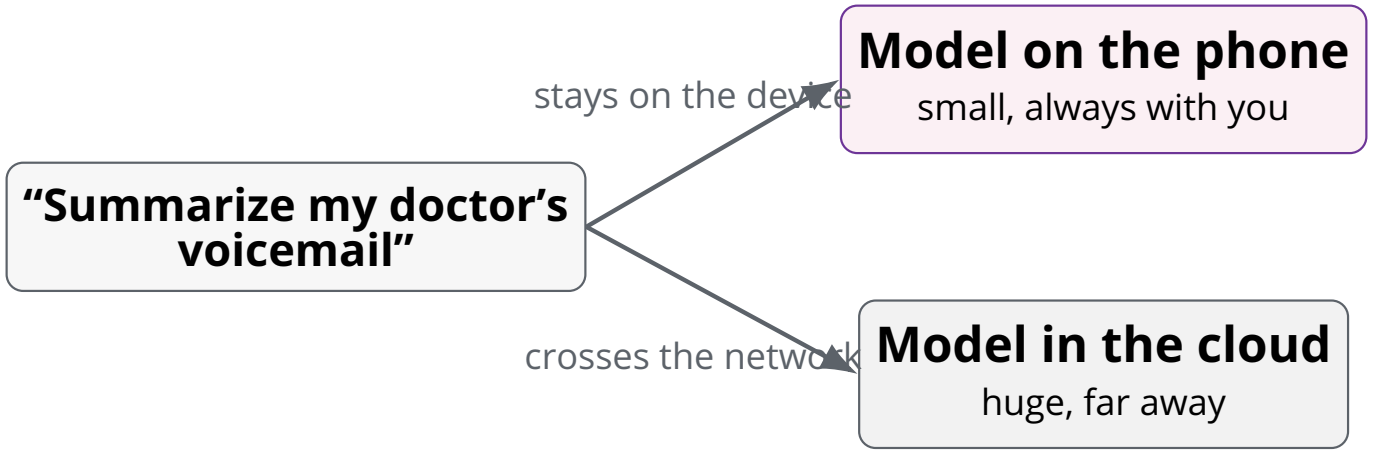


One question, two places to answer it



Where should the model run?

One question, two places to answer it



Where should the model run?

Both answers ship today: your phone has a model in it, and a data center has a bigger one. The interesting question is **which work goes where — and why**.

Today's route

1. Why run AI on the device at all?
2. The device's **budget**: memory, bandwidth, energy
3. **Quantization** — fewer bits per weight
4. **Mixture of Experts** — don't touch every weight
5. The **KV cache** — the memory that grows as you chat
6. **Split** and **collaborative** inference — cut the model in two
7. **Model routing** — send the hard questions away

Today's route

1. Why run AI on the device at all?
2. The device's **budget**: memory, bandwidth, energy
3. **Quantization** — fewer bits per weight
4. **Mixture of Experts** — don't touch every weight
5. The **KV cache** — the memory that grows as you chat
6. **Split** and **collaborative** inference — cut the model in two
7. **Model routing** — send the hard questions away

One budget, five responses to it.

After the lecture: **Lex** presents **MobileMoE** (stop 4, on real phones).
Wednesday: **D2MoE** — experts that don't fit.

Four reasons to stay on the device

Latency no network round trip — the answer starts now, not 100 ms from now

Four reasons to stay on the device

Latency no network round trip — the answer starts now, not 100 ms from now

Privacy the voicemail never leaves your phone

Four reasons to stay on the device

Latency no network round trip — the answer starts now, not 100 ms from now

Privacy the voicemail never leaves your phone

Availability works on a plane, in a basement, in an outage

Four reasons to stay on the device

Latency no network round trip — the answer starts now, not 100 ms from now

Privacy the voicemail never leaves your phone

Availability works on a plane, in a basement, in an outage

Cost a billion users $\times$ free queries = someone's GPU bill; your phone's silicon is already paid for

Four reasons to stay on the device

Latency no network round trip — the answer starts now, not 100 ms from now

Privacy the voicemail never leaves your phone

Availability works on a plane, in a basement, in an outage

Cost a billion users $\times$ free queries = someone's GPU bill; your phone's silicon is already paid for

On-device doesn't need to beat the cloud model —
it needs to be **good enough** for the easy majority.

You already run models all day

- Keyboard: next-word prediction, autocorrect
- Camera: night mode, portrait segmentation — on the NPU (neural processing unit)
- “Hey Siri” / “Hey Google”: the wake word is detected **locally** — nobody streams your mic to a server all day
- Watch: heart-rhythm classification on the wrist
- Live translate, live captions: work in airplane mode

Apple, “Apple Intelligence Foundation Language Models,” 2024; Google, Gemini Nano on Pixel, 2023–2026.

You already run models all day

- Keyboard: next-word prediction, autocorrect
- Camera: night mode, portrait segmentation — on the NPU (neural processing unit)
- “Hey Siri” / “Hey Google”: the wake word is detected **locally** — nobody streams your mic to a server all day
- Watch: heart-rhythm classification on the wrist
- Live translate, live captions: work in airplane mode

New since ∼2024: **full LLMs** — Apple runs a ∼3B-parameter model on the phone and routes heavy requests to its cloud; Pixel ships **Gemini Nano** for summaries.

Apple, “Apple Intelligence Foundation Language Models,” 2024; Google, Gemini Nano on Pixel, 2023–2026.

The cloud’s counterargument

- The best model **doesn’t fit**: frontier models are hundreds of GB of weights

The cloud's counterargument

- The best model **doesn't fit**: frontier models are hundreds of GB of weights
- A data-center GPU is **shared**: thousands of users batch on one card; your NPU serves one user, then idles

The cloud's counterargument

- The best model **doesn't fit**: frontier models are hundreds of GB of weights
- A data-center GPU is **shared**: thousands of users batch on one card; your NPU serves one user, then idles
- One copy in the cloud: always updated, measured, guarded

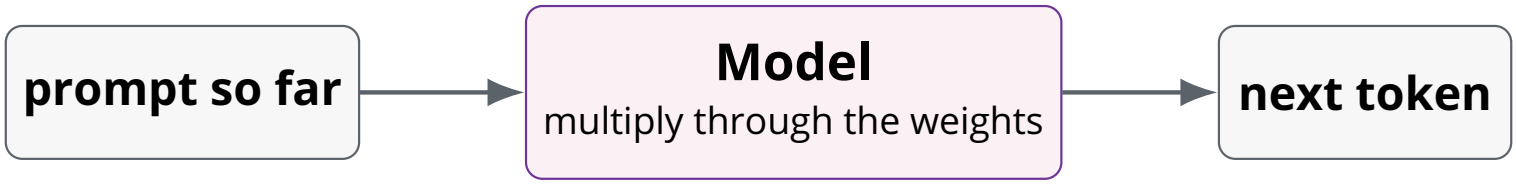
The cloud's counterargument

- The best model **doesn't fit**: frontier models are hundreds of GB of weights
- A data-center GPU is **shared**: thousands of users batch on one card; your NPU serves one user, then idles
- One copy in the cloud: always updated, measured, guarded

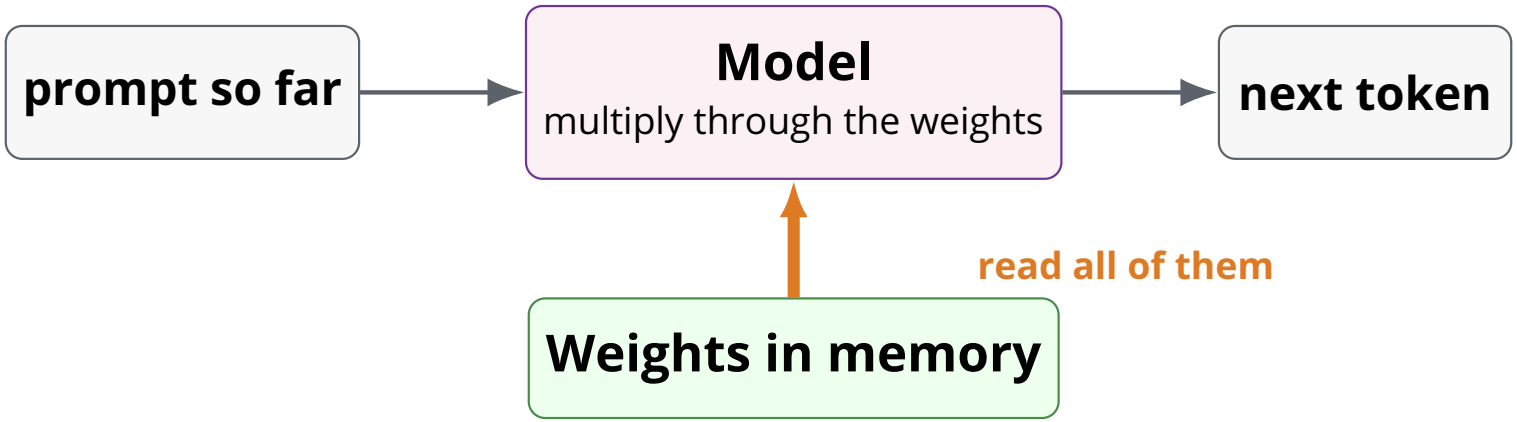
Not "device vs. cloud" — a **partition**: which part of the work runs where?

To partition well, we first need the device's budget.

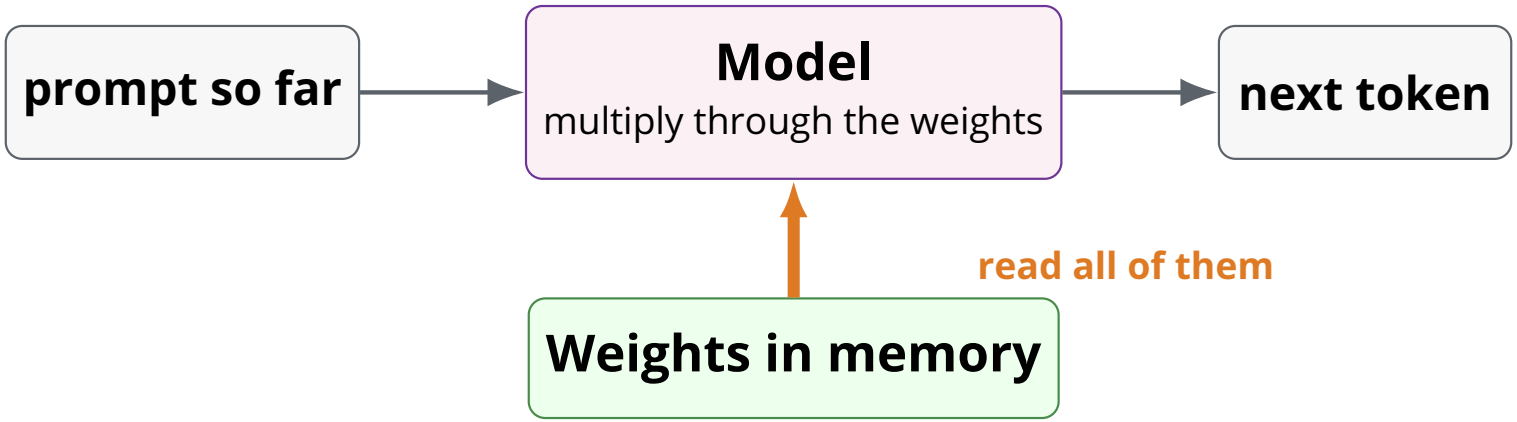
What one token costs



What one token costs

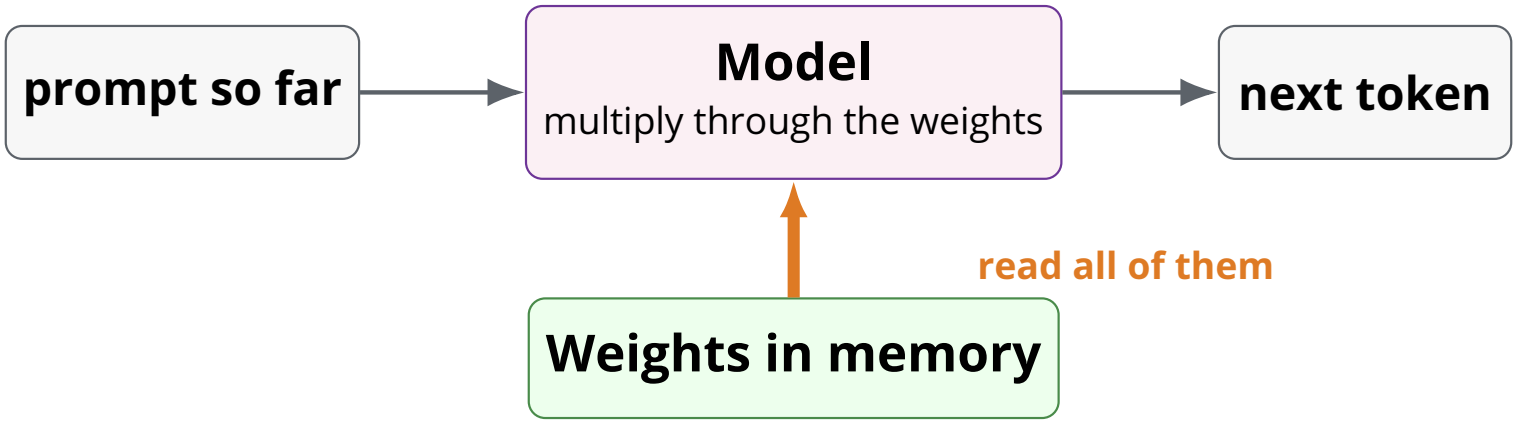


What one token costs



…and again for the next token, and the next

What one token costs



Two numbers rule on-device LLMs:
bytes the weights occupy and **bytes/second the memory delivers**.

The budget, by device

	Phone	Laptop	Cloud GPU (H10
Memory for the model	a few GB	tens of GB	80 GB HBM
Memory bandwidth	∼70 GB/s	100–400 GB/s	3,350 GB/s
Sustained power	a few W	tens of W	∼700 W
Serving	you	you	thousands of use

Rounded, representative figures: LPDDR5X-class phones (∼2024–26); Apple M-series unified memory; NVIDIA H100 SXM (80 GB HBM3, 3.35 TB/s).

The budget, by device

	Phone	Laptop	Cloud GPU (H10
Memory for the model	a few GB	tens of GB	80 GB HBM
Memory bandwidth	∼70 GB/s	100–400 GB/s	3,350 GB/s
Sustained power	a few W	tens of W	∼700 W
Serving	you	you	thousands of use

The GPU's memory is ∼**50**× **faster** than the phone's. Same math, a very different budget.

Rounded, representative figures: LPDDR5X-class phones (∼2024–26); Apple M-series unified memory; NVIDIA H100 SXM (80 GB HBM3, 3.35 TB/s).

Do the math: a 7B model on a phone

Fit?

7B weights $\times$ 2 bytes (FP16) $=$ **14 GB** — the phone has **a few GB** to spare. **Doesn't fit.**

Do the math: a 7B model on a phone

Fit?

7B weights $\times$ 2 bytes (FP16) $=$ **14 GB** — the phone has **a few GB** to spare. **Doesn't fit.**

Fast enough, even if it fit?

every token reads 14 GB; at 70 GB/s: $\frac{70\ \text{GB/s}}{14\ \text{GB}} = \mathbf{5\ tokens/s}$ at best — reading speed, zero margin.

Do the math: a 7B model on a phone

Fit?

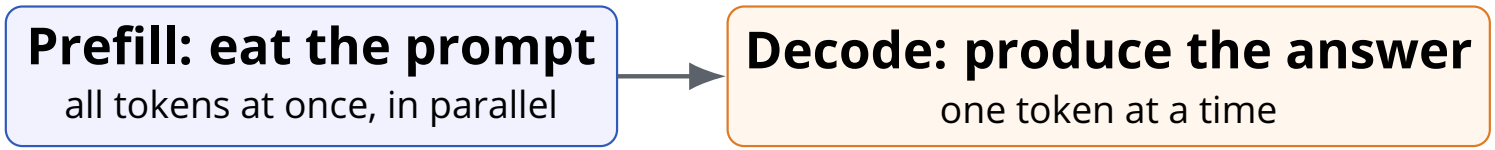
7B weights $\times$ 2 bytes (FP16) $=$ **14 GB** — the phone has **a few GB** to spare. **Doesn't fit.**

Fast enough, even if it fit?

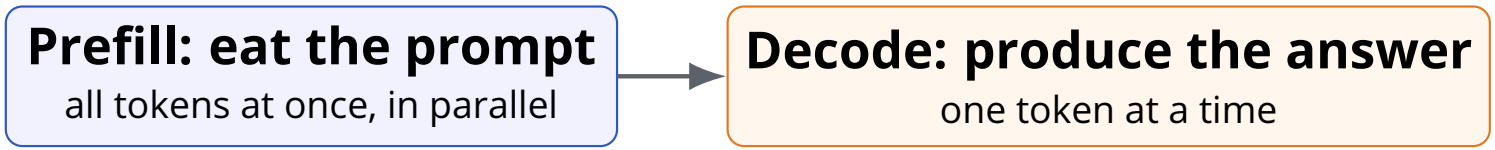
every token reads 14 GB; at 70 GB/s: $\frac{70\ \text{GB/s}}{14\ \text{GB}} = \textbf{5 tokens/s}$ at best — reading speed, zero margin.

Both walls are made of **bytes**.
Every trick today is a way to move fewer of them.

Two phases, two bottlenecks



Two phases, two bottlenecks



compute-bound — the pause before the first word

bandwidth-bound — the streaming speed after it

Two phases, two bottlenecks

Prefill: eat the prompt
all tokens at once, in parallel

→

Decode: produce the answer
one token at a time

compute-bound — the pause before the first word

bandwidth-bound — the streaming speed after it

You have felt both: the **pause**, then the **stream**.
Different walls, different fixes.

And the battery agrees

- Moving a byte from DRAM costs **far more energy** than computing with it once it has arrived
- No fan, held in a hand: sustained generation $\rightarrow$ heat $\rightarrow$ **throttling**

And the battery agrees

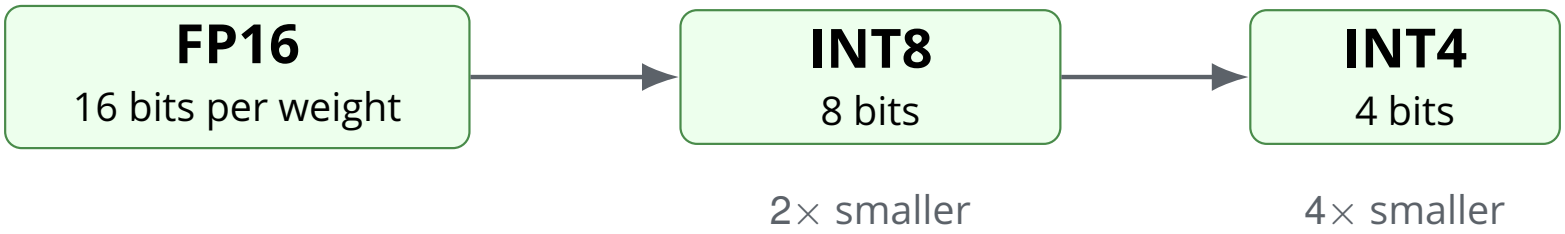
- Moving a byte from DRAM costs **far more energy** than computing with it once it has arrived
- No fan, held in a hand: sustained generation $\rightarrow$ heat $\rightarrow$ **throttling**

The bandwidth wall and the battery wall are the **same wall**: bytes moved. One fix serves both.

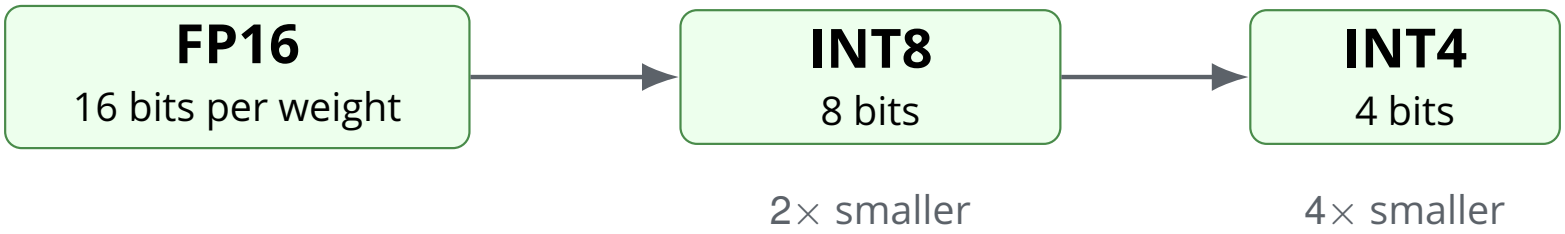
Trick 1: store each weight in fewer bits

FP16
16 bits per weight

Trick 1: store each weight in fewer bits

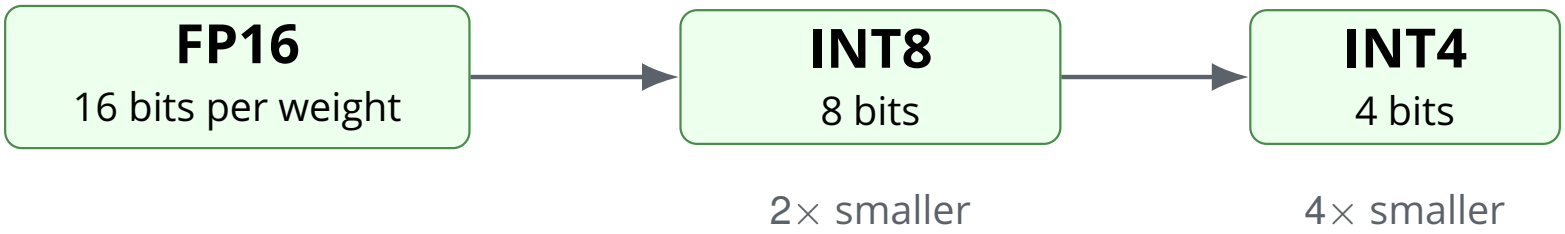


Trick 1: store each weight in fewer bits



Round each weight onto a coarse grid; keep one **scale** per group of weights to map back. No retraining — just re-encoding the numbers.

Trick 1: store each weight in fewer bits



Round each weight onto a coarse grid; keep one **scale** per group of weights to map back. No retraining — just re-encoding the numbers.

Like rounding every price to the nearest dollar:
the catalog shrinks, most totals barely move.

The same math at 4 bits

Fit?

7B weights $\times$ 0.5 byte (INT4) $=$ **3.5 GB** — **fits** next to the OS.

The same math at 4 bits

Fit?

7B weights $\times$ 0.5 byte (INT4) $=$ **3.5 GB** — **fits** next to the OS.

Fast enough?

every token now reads 3.5 GB: $\frac{70 \text{ GB/s}}{3.5 \text{ GB}} = \textbf{20 tokens/s}$ — a comfortable chat.

The same math at 4 bits

Fit?

7B weights $\times$ 0.5 byte (INT4) $=$ **3.5 GB** — **fits** next to the OS.

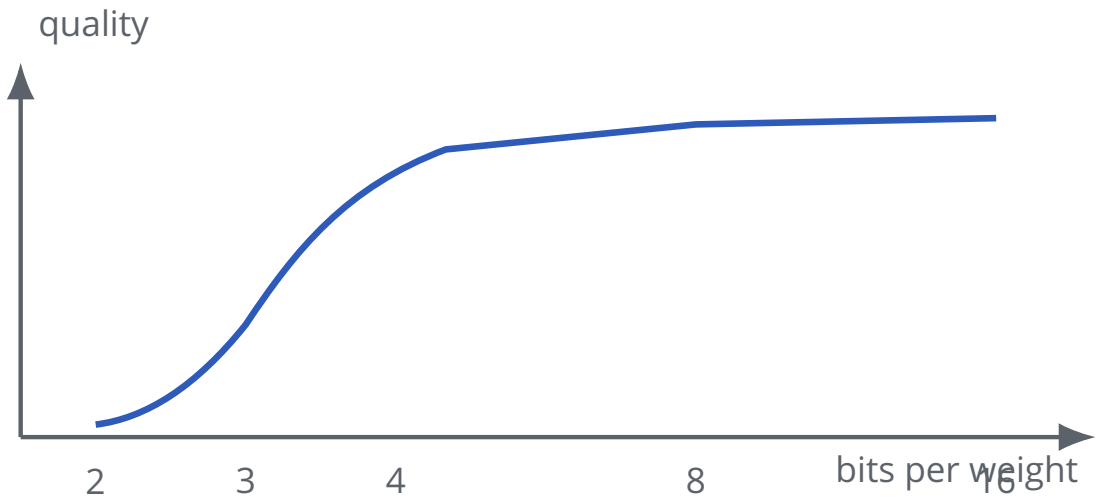
Fast enough?

every token now reads 3.5 GB: $\frac{70 \text{ GB/s}}{3.5 \text{ GB}} = \textbf{20 tokens/s}$ — a comfortable chat.

Decode is bandwidth-bound, so 4$\times$ fewer bytes
$\approx$ 4$\times$ faster streaming. Not a paper number — you feel it.

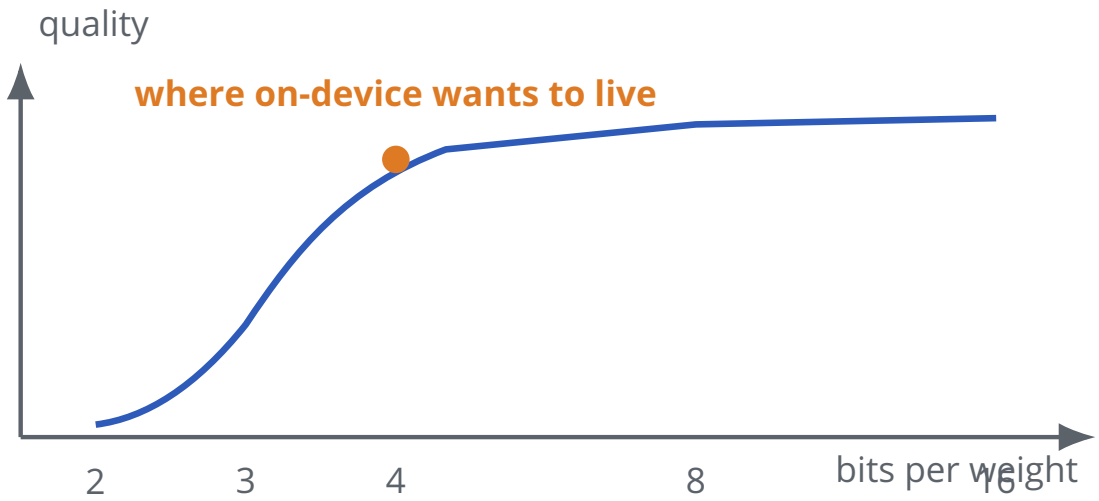
Why 4 bits is not free

A few **outlier** weights stretch the grid; the many small weights collapse onto a few levels.



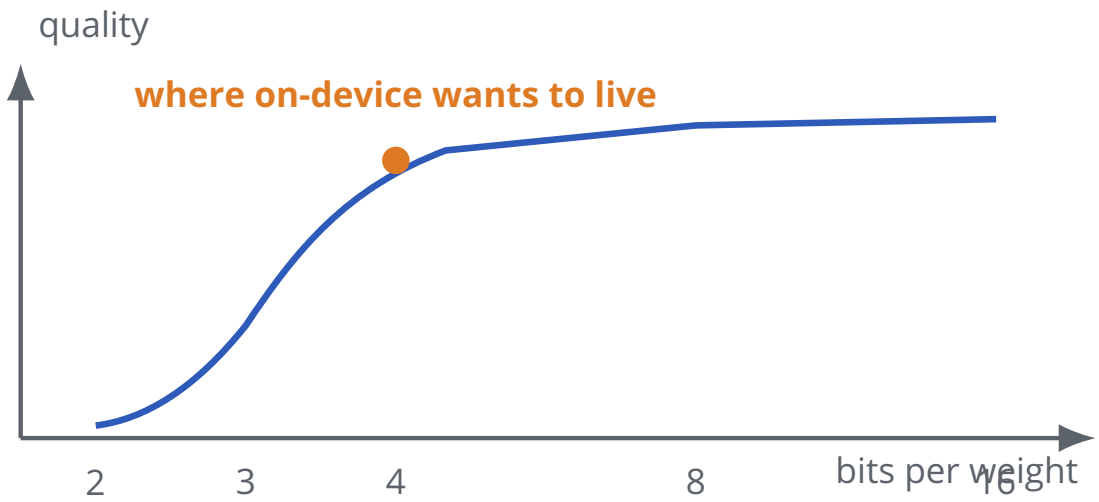
Why 4 bits is not free

A few **outlier** weights stretch the grid; the many small weights collapse onto a few levels.



Why 4 bits is not free

A few **outlier** weights stretch the grid; the many small weights collapse onto a few levels.



Flat from 16 to 8 bits; the cliff starts right where the phone needs to be. Naive rounding falls off it.

How real quantizers stay on the ledge

1. **Calibrate**: run a few hundred sample inputs; pick each grid to minimize the error that actually appears in the output

Reading, if you want the algorithms: GPTQ (round, then correct the remaining weights) and AWQ (rescale the salient channels before rounding).

Frantar et al., “GPTQ,” ICLR 2023; Lin et al., “AWQ,” MLSys 2024 (Best Paper).

How real quantizers stay on the ledge

1. **Calibrate**: run a few hundred sample inputs; pick each grid to minimize the error that actually appears in the output
2. **Protect the outliers**: keep the ∼1% extreme channels in higher precision — or rescale them so the shared grid fits the rest

Reading, if you want the algorithms: GPTQ (round, then correct the remaining weights) and AWQ (rescale the salient channels before rounding).

Frantar et al., "GPTQ," ICLR 2023; Lin et al., "AWQ," MLSys 2024 (Best Paper).

How real quantizers stay on the ledge

1. **Calibrate**: run a few hundred sample inputs; pick each grid to minimize the error that actually appears in the output
2. **Protect the outliers**: keep the ∼1% extreme channels in higher precision — or rescale them so the shared grid fits the rest
3. **Weights first**: activations change with every input and are harder — phones mostly ship weights-only INT4

Reading, if you want the algorithms: GPTQ (round, then correct the remaining weights) and AWQ (rescale the salient channels before rounding).

Frantar et al., "GPTQ," ICLR 2023; Lin et al., "AWQ," MLSys 2024 (Best Paper).

Quantization, in the wild

- Apple's on-device model: $\sim$**3.7 bits** per weight on average (mixed 2-/4-bit)
- Gemini Nano, and every 4-bit model file you can run with `llama.cpp`: same idea

Apple, "Apple Intelligence Foundation Language Models," 2024.

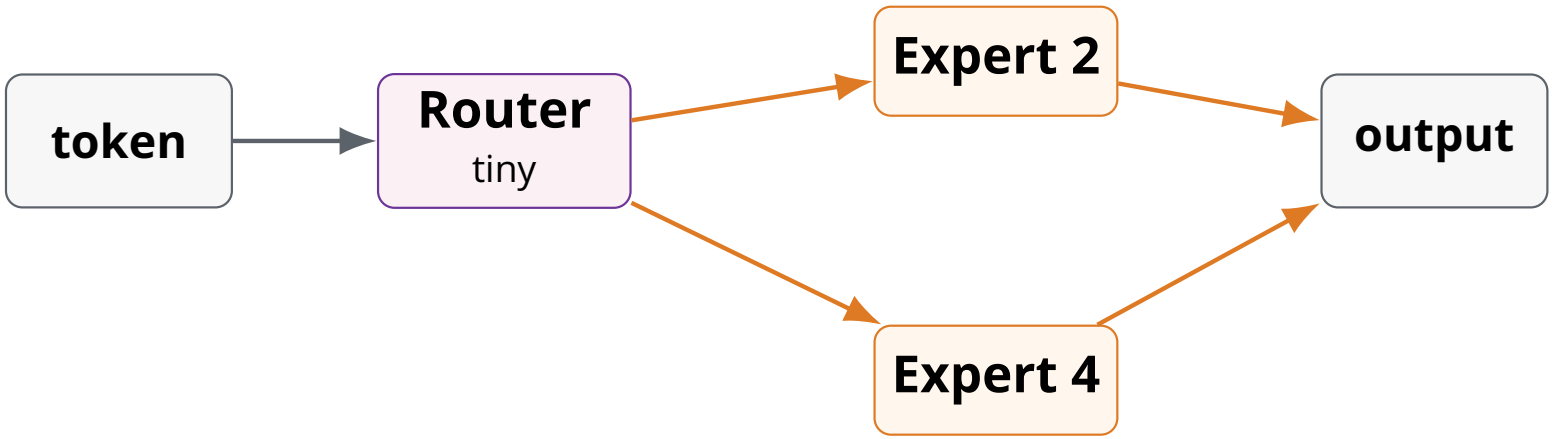
Quantization, in the wild

- Apple's on-device model: $\sim$**3.7 bits** per weight on average (mixed 2-/4-bit)
- Gemini Nano, and every 4-bit model file you can run with `llama.cpp`: same idea

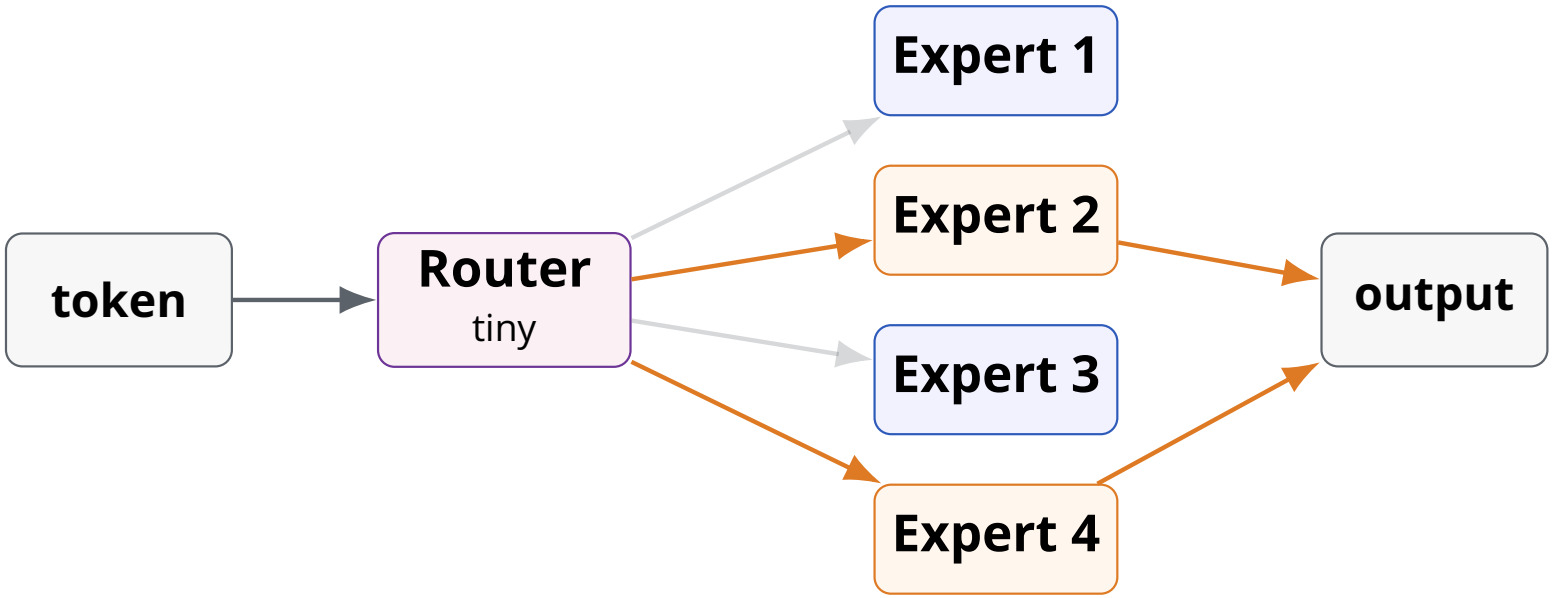
Apple, "Apple Intelligence Foundation Language Models," 2024.

Quantization makes every byte smaller.
Next trick: **don't touch most bytes at all**.

Trick 2: many experts, use a few

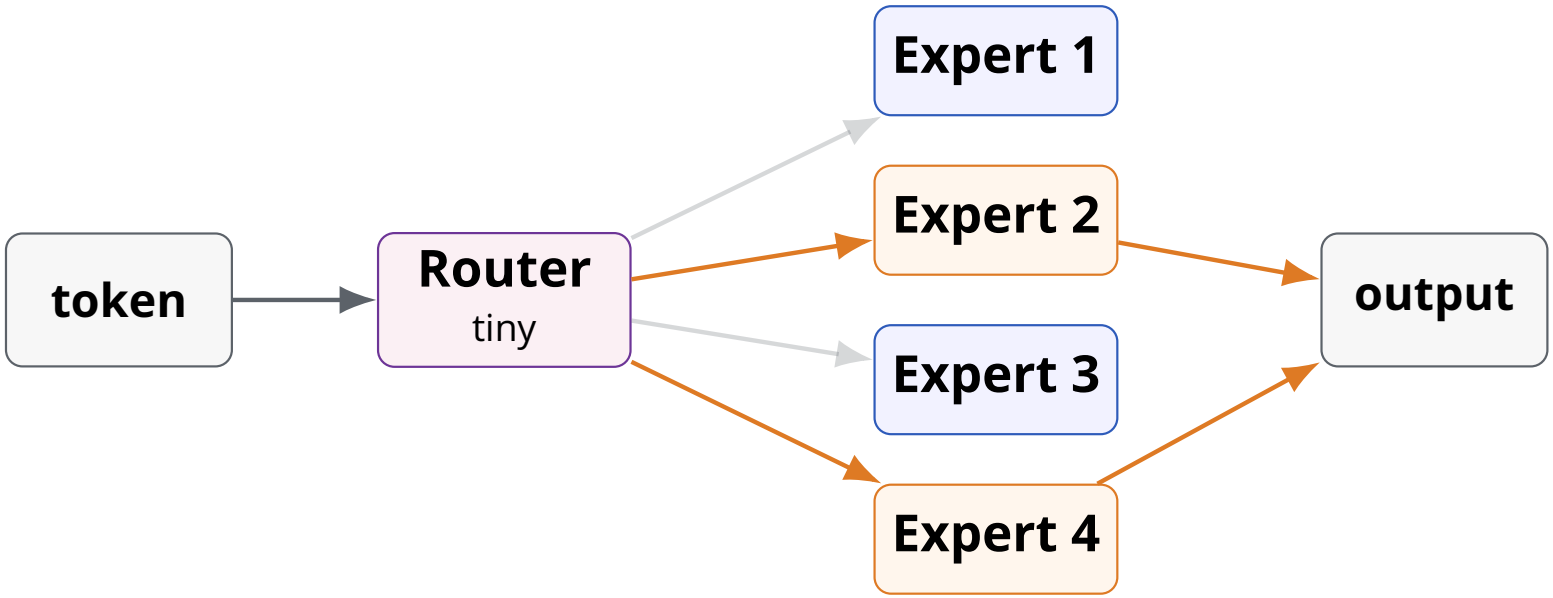


Trick 2: many experts, use a few



the dark experts hold weights, but this token never reads them

Trick 2: many experts, use a few



the dark experts hold weights, but this token never reads them

Total capacity grows; **active** weights per token stay small.
You store a big model but read a small one.

Why every frontier model went MoE

- DeepSeek-V3: **671B** parameters total, ∼**37B active** per token
- Llama 4, Qwen3, gpt-oss: all MoE — the 2025 frontier default

DeepSeek-AI, “DeepSeek-V3 Technical Report,” 2024.

Why every frontier model went MoE

- DeepSeek-V3: **671B** parameters total, ∼**37B active** per token
- Llama 4, Qwen3, gpt-oss: all MoE — the 2025 frontier default

The data center's trade: memory is **plentiful** (80 GB HBM per GPU, many GPUs), compute per token is the scarce thing. MoE spends memory to save compute.

DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024.

Why every frontier model went MoE

- DeepSeek-V3: **671B** parameters total, ∼**37B active** per token
- Llama 4, Qwen3, gpt-oss: all MoE — the 2025 frontier default

The data center's trade: memory is **plentiful** (80 GB HBM per GPU, many GPUs), compute per token is the scarce thing. MoE spends memory to save compute.

MoE buys capacity **with memory** instead of compute — perfect for a data center.

DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024.

A phone flips the trade

- Idle experts still **occupy RAM** — the resource the phone lacks most
- Every weight touched is **bandwidth paid** — and each token picks different experts
- One user, no batching — dark experts earn nothing while they sit

A phone flips the trade

- Idle experts still **occupy RAM** — the resource the phone lacks most
- Every weight touched is **bandwidth paid** — and each token picks different experts
- One user, no batching — dark experts earn nothing while they sit

Same trick, opposite budget:
what should an MoE look like on a phone?

MobileMoE: the phone-shaped answer (teaser)

Meta fitted an MoE **scaling law on real smartphones** and asked: at a fixed weight-memory budget, what shape wins?

MobileMoE: Scaling On-Device Mixture of Experts, Meta, arXiv, May 2026.

MobileMoE: the phone-shaped answer (teaser)

Meta fitted an MoE **scaling law on real smartphones** and asked: at a fixed weight-memory budget, what shape wins?

- **60 routed experts + 1 shared**, route to **top-4** — fine-grained, moderately sparse
- At equal weight memory vs. dense: **1.8–3.8**× faster prefill, **2.2–3.4**× faster decode

MobileMoE: Scaling On-Device Mixture of Experts, Meta, arXiv, May 2026.

MobileMoE: the phone-shaped answer (teaser)

Meta fitted an MoE **scaling law on real smartphones** and asked: at a fixed weight-memory budget, what shape wins?

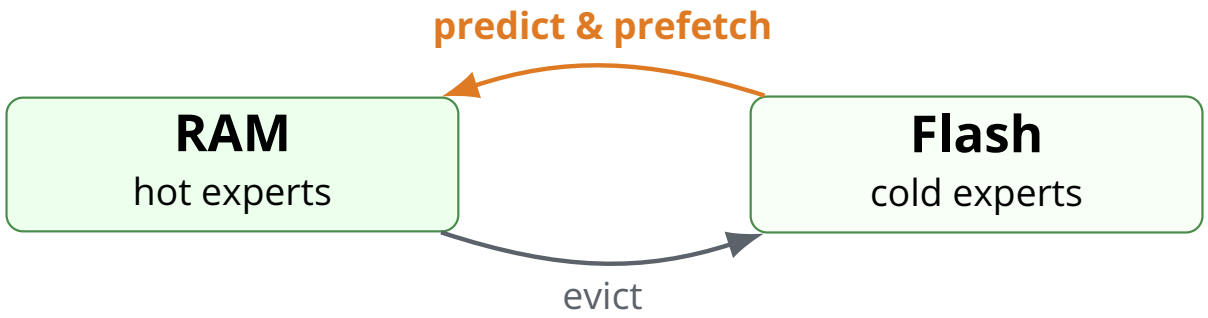
- **60 routed experts + 1 shared**, route to **top-4** — fine-grained, moderately sparse
- At equal weight memory vs. dense: **1.8–3.8**× faster prefill, **2.2–3.4**× faster decode

Capacity with small active bytes per token
— MoE reshaped for the phone's budget.

The how — the scaling law, the shared expert, the measurements — is **Lex's talk**, right after this lecture.

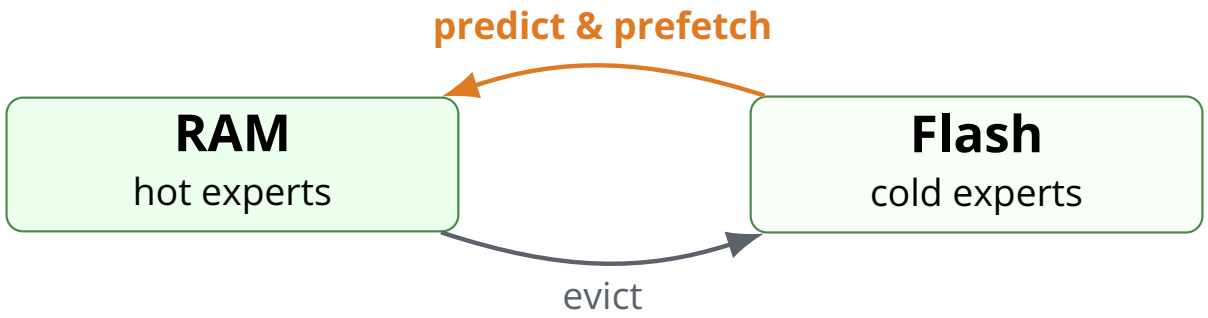
MobileMoE: Scaling On-Device Mixture of Experts, Meta, arXiv, May 2026.

The loose end: experts that don’t fit



Guess the next experts early and right, and decode never notices.
Guess wrong, and it stalls on flash.

The loose end: experts that don’t fit

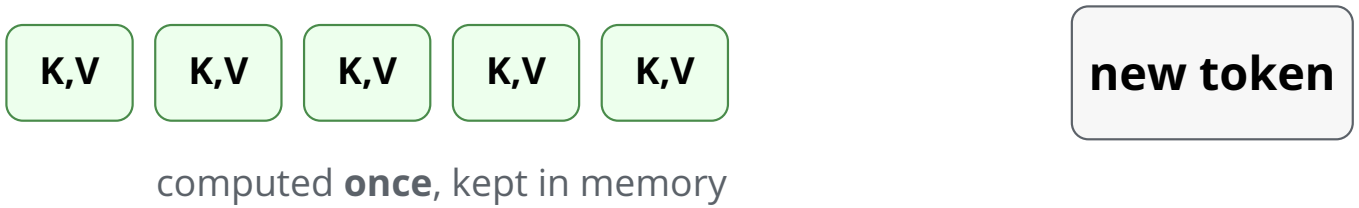


Guess the next experts early and right, and decode never notices.
Guess wrong, and it stalls on flash.

Which experts to keep, spill, and prefetch —
that is **D2MoE**, Wednesday 11/4.

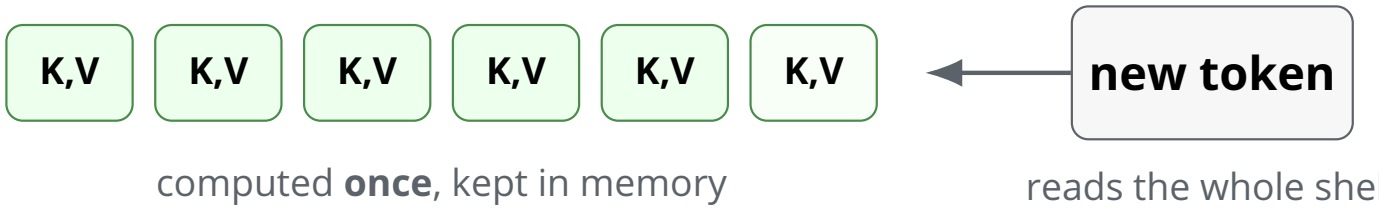
Trick 3's target: the cache you didn't know you had

Attention: the new token compares against a **key** and **value** for every previous token (PyTorch lecture, 10/19).



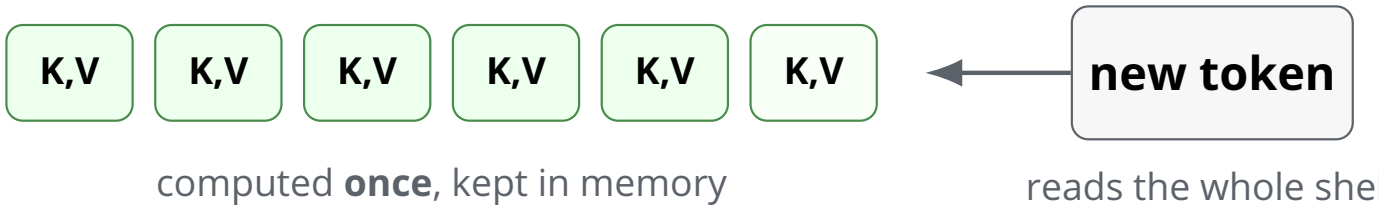
Trick 3's target: the cache you didn't know you had

Attention: the new token compares against a **key** and **value** for every previous token (PyTorch lecture, 10/19).



Trick 3's target: the cache you didn't know you had

Attention: the new token compares against a **key** and **value** for every previous token (PyTorch lecture, 10/19).



Cache each token's K,V instead of recomputing the whole history
every step — pure speed-for-memory. So far, free lunch.

The cache grows as you chat

Per token: $2 \times$ layers $\times$ KV heads $\times$ head dim $\times$ bytes

The cache grows as you chat

Per token: $2 \times$ layers $\times$ KV heads $\times$ head dim $\times$ bytes

A classic 7B model (32 layers, 32 heads $\times$ 128, FP16):

$$2 \times 32 \times 32 \times 128 \times 2\ \text{B} \approx \textbf{0.5 MB per token}$$

The cache grows as you chat

Per token: $2 \times$ layers $\times$ KV heads $\times$ head dim $\times$ bytes

A classic 7B model (32 layers, 32 heads $\times$ 128, FP16):

$$2 \times 32 \times 32 \times 128 \times 2\ \text{B} \approx \textbf{0.5 MB per token}$$

A 4,000-token chat: $4{,}000 \times 0.5$ MB $\approx$ **2 GB** — as big as the whole 4-bit model we just squeezed to 3.5 GB.

The cache grows as you chat

Per token: $2 \times$ layers $\times$ KV heads $\times$ head dim $\times$ bytes

A classic 7B model (32 layers, 32 heads $\times$ 128, FP16):

$$2 \times 32 \times 32 \times 128 \times 2 \text{ B} \approx \textbf{0.5 MB per token}$$

A 4,000-token chat: $4{,}000 \times 0.5$ MB $\approx$ **2 GB** — as big as the whole 4-bit model we just squeezed to 3.5 GB.

Weights are fixed; **the cache grows**.
Long chats are a memory problem, not a compute problem.

Putting the cache on a diet

Share it grouped-query attention: many query heads, few KV heads — 8 instead of 32 is a built-in 4×

Putting the cache on a diet

Share it grouped-query attention: many query heads, few KV heads — 8 instead of 32 is a built-in 4×

Quantize it the same 4-bit trick, applied to K,V

Putting the cache on a diet

Share it grouped-query attention: many query heads, few KV heads — 8 instead of 32 is a built-in 4×

Quantize it the same 4-bit trick, applied to K,V

Evict drop or summarize old, rarely-attended tokens — the model forgets the middle of the chat

Putting the cache on a diet

Share it grouped-query attention: many query heads, few KV heads — 8 instead of 32 is a built-in 4×

Quantize it the same 4-bit trick, applied to K,V

Evict drop or summarize old, rarely-attended tokens — the model forgets the middle of the chat

Each row trades a little context fidelity for memory.
Together: how a phone holds a long conversation at all.

Scorecard: three tricks, one currency

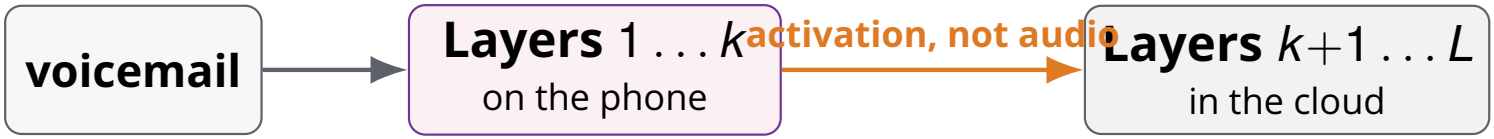
- Quantization: every byte **smaller** (14 GB $\rightarrow$ 3.5 GB)
- MoE: most bytes **never touched** per token
- KV diet: the **growing** bytes, capped

Scorecard: three tricks, one currency

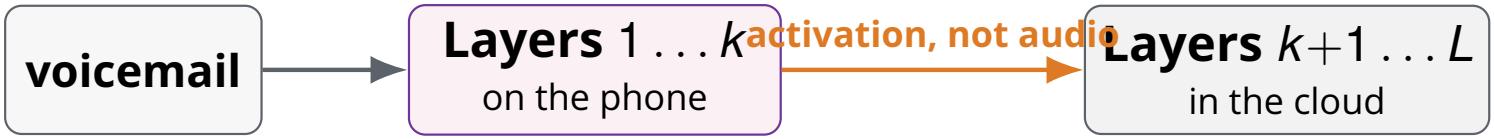
- Quantization: every byte **smaller** (14 GB $\rightarrow$ 3.5 GB)
- MoE: most bytes **never touched** per token
- KV diet: the **growing** bytes, capped

All three accept the phone's budget.
The last two tricks **refuse it**: change where the job runs.

Trick 4: cut the model in two

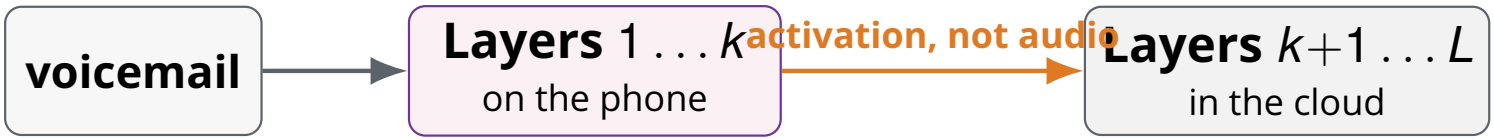


Trick 4: cut the model in two



what crosses is a representation — Class 17's move, reused

Trick 4: cut the model in two



what crosses is a representation — Class 17's move, reused

The raw data never leaves; the network carries an embedding.
Split point k is a dial, not a fact.

When the cut wins — and when it doesn't

Wins when

- the input is big and sensory (audio, video), the representation small
- raw data must not leave the device

When the cut wins — and when it doesn't

Wins when

- the input is big and sensory (audio, video), the representation small
- raw data must not leave the device

Loses when

- the activation is bigger than the input — a text prompt is a few KB; hidden states aren't
- the round trip we fled is back in the loop
- the cloud is gone — a split model is headless offline

When the cut wins — and when it doesn't

Wins when

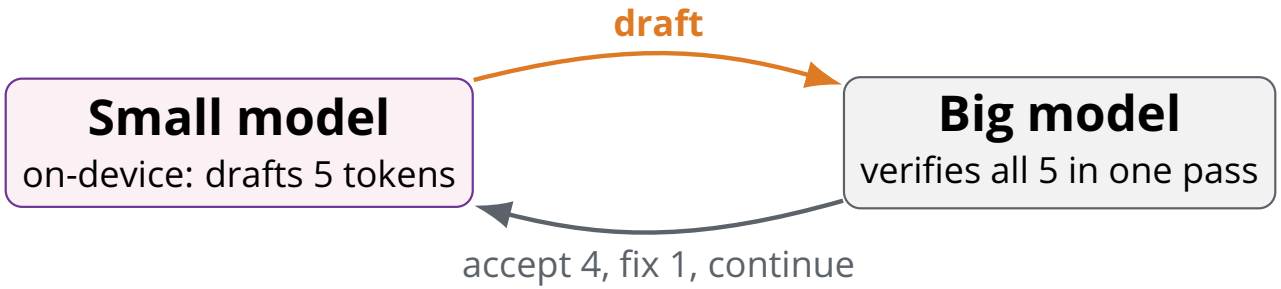
- the input is big and sensory (audio, video), the representation small
- raw data must not leave the device

Loses when

- the activation is bigger than the input — a text prompt is a few KB; hidden states aren't
- the round trip we fled is back in the loop
- the cloud is gone — a split model is headless offline

Split big-sensory, small-representation jobs.
Don't split a text chat.

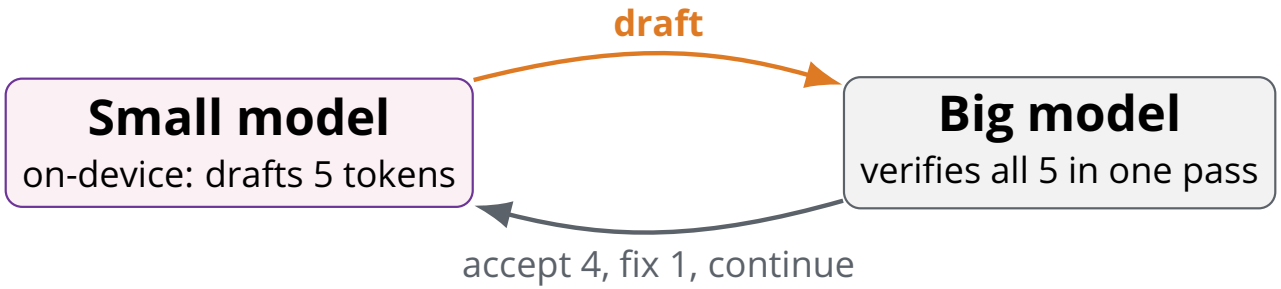
Or collaborate: draft small, verify big



Checking *k* given tokens is **one** parallel pass — not *k* slow decode steps. Easy words are easy: most drafts survive.

Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” ICML 2023.

Or collaborate: draft small, verify big



Checking *k* given tokens is **one** parallel pass — not *k* slow decode steps. Easy words are easy: most drafts survive.

The big model runs once per several tokens;
the small one hides the distance. Speed without losing the big model's judgment.

Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML 2023.

The boundary is a dial

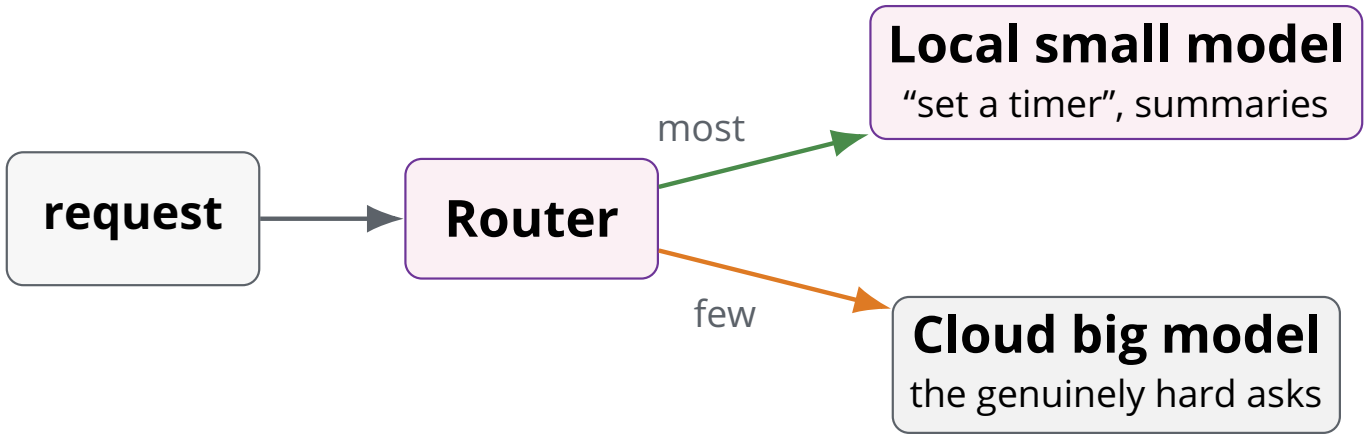


The boundary is a dial

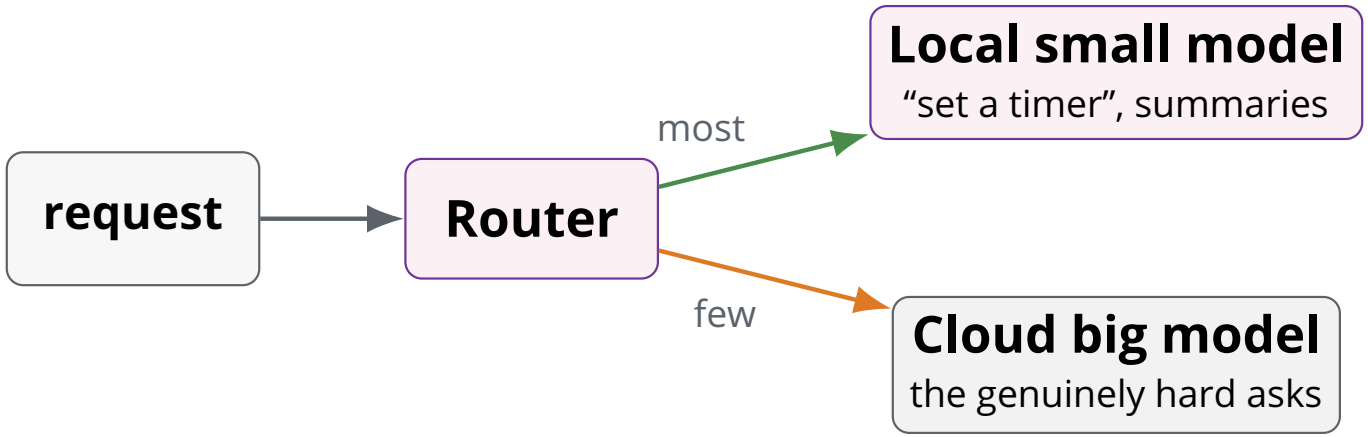


Four settings on one dial.
Last question: **who turns the dial**, per request?

Trick 5: send the question, not every question



Trick 5: send the question, not every question



Average cost of the small model,
worst-case quality of the big one.

How the router decides

1. **Rules**: task type and length — timers local, 30-page analysis cloud

Apple, “Introducing Apple’s On-Device and Server Foundation Models,” 2024.

How the router decides

1. **Rules**: task type and length — timers local, 30-page analysis cloud
2. **Confidence**: answer locally; escalate when the small model's own token probabilities look unsure

Apple, "Introducing Apple's On-Device and Server Foundation Models," 2024.

How the router decides

1. **Rules**: task type and length — timers local, 30-page analysis cloud
2. **Confidence**: answer locally; escalate when the small model's own token probabilities look unsure
3. **A learned router**: trained on requests where people preferred the big model's answer

Apple, "Introducing Apple's On-Device and Server Foundation Models," 2024.

How the router decides

1. **Rules**: task type and length — timers local, 30-page analysis cloud
2. **Confidence**: answer locally; escalate when the small model's own token probabilities look unsure
3. **A learned router**: trained on requests where people preferred the big model's answer

This is the shipped design: Apple routes between its on-device model and Private Cloud Compute; Pixel between Gemini Nano and cloud Gemini.

Apple, "Introducing Apple's On-Device and Server Foundation Models," 2024.

The assistant was never one model

What answers you is a **system**: a router, a quantized on-device model, a KV cache on a diet, and an MoE in the cloud — with split points and drafts in between.

The assistant was never one model

What answers you is a **system**: a router, a quantized on-device model, a KV cache on a diet, and an MoE in the cloud — with split points and drafts in between.

"Where should the model run?" —
the shipped answer is: **everywhere, each part where it is cheapest.**

One map of today

The wall	The trick
Weights don’t fit	Quantization: fewer bits per weight
Reading them is slow	Quantization again: fewer bytes per token
Need more capacity	MoE: store big, read small
Long chats eat RAM	KV cache: share, quantize, evict
Still too big	Split: send the representation
Cloud too slow/far	Collaborate: draft small, verify big
Most queries are easy	Route: local first, escalate the hard

One map of today

The wall	The trick
Weights don’t fit	Quantization: fewer bits per weight
Reading them is slow	Quantization again: fewer bytes per token
Need more capacity	MoE: store big, read small
Long chats eat RAM	KV cache: share, quantize, evict
Still too big	Split: send the representation
Cloud too slow/far	Collaborate: draft small, verify big
Most queries are easy	Route: local first, escalate the hard

One budget — bytes in memory, bytes moved, joules —
and every trick is a way to spend less of it.

If you remember three things

1. On a phone, LLM decode is **bandwidth-bound**: tokens/s $\approx$ bandwidth $\div$ bytes per token
2. Quantization, MoE, and the KV diet are all the **same move**: fewer bytes
3. Device vs. cloud is a **dial** — split, draft, route — not a wall

Next

- **Now**: Lex presents **MobileMoE** (15 min + 5 min Q&A)
- **Wednesday 11/4**: D2MoE — experts that don't fit in memory
- HW4 due **11/4, 11:59 p.m. ET**; final project released 11/4

Questions?